

APIs and HTTP

What is an API?

- API stands for **Application Programming Interface**: an API is a defined boundary between two pieces of software so they can communicate without either one needing to know how the other is implemented.

What is an API?

- API stands for **Application Programming Interface**: an API is a defined boundary between two pieces of software so they can communicate without either one needing to know how the other is implemented.
- You have already used APIs. When you call `numpy.mean(data)`, you are using NumPy's API: a named function with documented inputs and outputs.

What is an API?

- API stands for **Application Programming Interface**: an API is a defined boundary between two pieces of software so they can communicate without either one needing to know how the other is implemented.
- You have already used APIs. When you call `numpy.mean(data)`, you are using NumPy's API: a named function with documented inputs and outputs.
- A **web API** (often called a **REST API**) exposes **endpoints** — URLs that accept requests and return responses. You send a request describing what you want; the server sends back a response, often as JSON data.

JSON (JavaScript Object Notation)

- Text format for structured data.

JSON (JavaScript Object Notation)

- Text format for structured data.
- Universal format for web APIs.

JSON (JavaScript Object Notation)

- Text format for structured data.
- Universal format for web APIs.
- Easy format for a quick save/load that is human-readable.

JSON (JavaScript Object Notation)

- Text format for structured data.
- Universal format for web APIs.
- Easy format for a quick save/load that is human-readable.

JSON (JavaScript Object Notation)

- Text format for structured data.
- Universal format for web APIs.
- Easy format for a quick save/load that is human-readable.

```
{  
  "key": "age_vector",  
  "value": [25, 30, 35, 40, 45],  
  "demographics": {  
    "age": 30,  
    "education": "Bachelor's degree"  
  }  
}
```

Key Web API terminology

Term	Meaning
Endpoint	A URL that accepts a particular kind of request
Request	What you send: an HTTP method (GET, POST...), a URL, optional headers, and an optional body
Response	What the server returns: a status code and a body
HTTP method	Describes intent. GET retrieves data; POST sends a body and asks the server to process it.
Header	Key-value metadata on a request or response
Body	The payload of the request
API key	A secret string that authenticates you

GET requests from Python: weather data

The Open-Meteo API provides weather forecasts without registration or an API key. You build a URL with query parameters and read back JSON — a **GET** request.

GET requests from Python: weather data

The Open-Meteo API provides weather forecasts without registration or an API key. You build a URL with query parameters and read back JSON — a **GET** request.

Python's standard library can do this with `urllib.request` and `urllib.parse` (no extra packages).

GET requests from Python: weather data

The Open-Meteo API provides weather forecasts without registration or an API key. You build a URL with query parameters and read back JSON — a **GET** request.

Python's standard library can do this with `urllib.request` and `urllib.parse` (no extra packages).

- `urllib.parse.urlencode(query)` — Turns a mapping into `key=value&key=value`, with characters percent-encoded for use in a URL.

GET requests from Python: weather data

The Open-Meteo API provides weather forecasts without registration or an API key. You build a URL with query parameters and read back JSON — a **GET** request.

Python's standard library can do this with `urllib.request` and `urllib.parse` (no extra packages).

- `urllib.parse.urlencode(query)` — Turns a mapping into `key=value&key=value`, with characters percent-encoded for use in a URL.
- `urllib.request.urlopen(url)` — Performs an HTTP GET by default. Returns a response object; use it in a `with` block so the connection closes cleanly.

GET requests from Python: weather data

The Open-Meteo API provides weather forecasts without registration or an API key. You build a URL with query parameters and read back JSON — a **GET** request.

Python's standard library can do this with `urllib.request` and `urllib.parse` (no extra packages).

- `urllib.parse.urlencode(query)` — Turns a mapping into `key=value&key=value`, with characters percent-encoded for use in a URL.
- `urllib.request.urlopen(url)` — Performs an HTTP GET by default. Returns a response object; use it in a `with` block so the connection closes cleanly.
- `resp.read()` — Reads the response body as **bytes**.

GET requests from Python: weather data

The Open-Meteo API provides weather forecasts without registration or an API key. You build a URL with query parameters and read back JSON — a **GET** request.

Python's standard library can do this with `urllib.request` and `urllib.parse` (no extra packages).

- `urllib.parse.urlencode(query)` — Turns a mapping into `key=value&key=value`, with characters percent-encoded for use in a URL.
- `urllib.request.urlopen(url)` — Performs an HTTP GET by default. Returns a response object; use it in a `with` block so the connection closes cleanly.
- `resp.read()` — Reads the response body as **bytes**.
- `.decode("utf-8")` — Converts bytes to a **str** (JSON text is almost always UTF-8).

GET requests from Python: weather data

The Open-Meteo API provides weather forecasts without registration or an API key. You build a URL with query parameters and read back JSON — a **GET** request.

Python's standard library can do this with `urllib.request` and `urllib.parse` (no extra packages).

- `urllib.parse.urlencode(query)` — Turns a mapping into `key=value&key=value`, with characters percent-encoded for use in a URL.
- `urllib.request.urlopen(url)` — Performs an HTTP GET by default. Returns a response object; use it in a `with` block so the connection closes cleanly.
- `resp.read()` — Reads the response body as **bytes**.
- `.decode("utf-8")` — Converts bytes to a **str** (JSON text is almost always UTF-8).
- `json.loads(s)` — Parses a JSON string into Python objects (`dict`, `list`, numbers, `None` for JSON `null`).

Example GET request to Open-Meteo API

```
import json
import urllib.request
import urllib.parse

params = urllib.parse.urlencode({
    "latitude": 33.64054,
    "longitude": -117.83891,
    "current": "apparent_temperature,precipitation_probability",
})

endpoint = f"https://api.open-meteo.com/v1/forecast?{params}"

with urllib.request.urlopen(endpoint) as resp:
    data = json.loads(resp.read().decode("utf-8"))
```

Abbreviated JSON shape:

Response from Open-Meteo API

Abbreviated JSON shape:

```
{
  "latitude": 33.64054,
  "longitude": -117.83891,
  "current": {
    "time": "2026-05-03T00:00",
    "apparent_temperature": 17.7,
    "precipitation_probability": 0
  }
}
```

Response from Open-Meteo API

Abbreviated JSON shape:

```
{
  "latitude": 33.64054,
  "longitude": -117.83891,
  "current": {
    "time": "2026-05-03T00:00",
    "apparent_temperature": 17.7,
    "precipitation_probability": 0
  }
}
```

The pattern — build URL, open it, decode JSON, navigate nested **dicts** — is how many read-only web APIs work. Only the URL shape and response structure change.

Exercise: get weather data for your location

- Check the Open-Meteo API documentation on:
https://open-meteo.com/en/docs#hourly_parameter_definition

Exercise: get weather data for your location

- Check the Open-Meteo API documentation on:
https://open-meteo.com/en/docs#hourly_parameter_definition
- Our latitude and longitude are 33.64054 and -117.83891

Exercise: get weather data for your location

- Check the Open-Meteo API documentation on:
https://open-meteo.com/en/docs#hourly_parameter_definition
- Our latitude and longitude are 33.64054 and -117.83891
- Write a script to get the **relative_humidity_2m** right now

POST requests

For larger, complex payloads, use **POST**: data goes in the **body** as JSON, with a **Content-Type: application/json** header.

POST requests

For larger, complex payloads, use POST: data goes in the **body** as JSON, with a **Content-Type: application/json** header.

```
import json
import urllib.request
url = "https://api.example.com/v1/endpoint"
body = {"key": "value", "count": 3}
body_bytes = json.dumps(body).encode("utf-8")
req = urllib.request.Request(url, data=body_bytes,
                             headers={"Content-Type": "application/json"}, method="POST")
with urllib.request.urlopen(req) as resp:
    data = json.loads(resp.read().decode("utf-8"))
print(data)
```

POST requests as a function

For larger, complex payloads, use **POST**: data goes in the **body** as JSON, with a **Content-Type: application/json** header.

POST requests as a function

For larger, complex payloads, use POST: data goes in the **body** as JSON, with a **Content-Type: application/json** header.

```
import json
import urllib.request
URL = "https://api.example.com/v1/endpoint"
def post_request(url, body):
    body_bytes = json.dumps(body).encode("utf-8")
    req = urllib.request.Request(url, data=body_bytes,
                                headers={"Content-Type": "application/json"}, method="POST")
    resp = urllib.request.urlopen(req)
    return json.loads(resp.read().decode("utf-8"))
print(post_request(URL, {"key": "value", "count": 3}))
```

- The Gemini API is a web API that allows you to generate text using the Gemini model.

- The Gemini API is a web API that allows you to generate text using the Gemini model.
- First, we need to obtain a Gemini API key:

The Gemini API

- The Gemini API is a web API that allows you to generate text using the Gemini model.
- First, we need to obtain a Gemini API key:
 1. Open Google AI Studio and sign in.

- The Gemini API is a web API that allows you to generate text using the Gemini model.
- First, we need to obtain a Gemini API key:
 1. Open Google AI Studio and sign in.
 2. Use **Get API key** → **Create API key**.

The Gemini API

- The Gemini API is a web API that allows you to generate text using the Gemini model.
- First, we need to obtain a Gemini API key:
 1. Open Google AI Studio and sign in.
 2. Use **Get API key** → **Create API key**.
 3. Copy the key (a long alphanumeric string).

- The Gemini API is a web API that allows you to generate text using the Gemini model.
- First, we need to obtain a Gemini API key:
 1. Open Google AI Studio and sign in.
 2. Use **Get API key** → **Create API key**.
 3. Copy the key (a long alphanumeric string).
- Practice good **key hygiene**:

- The Gemini API is a web API that allows you to generate text using the Gemini model.
- First, we need to obtain a Gemini API key:
 1. Open Google AI Studio and sign in.
 2. Use **Get API key** → **Create API key**.
 3. Copy the key (a long alphanumeric string).
- Practice good **key hygiene**:
 - **Never** commit API keys in source files, notebooks, or screenshots you share.

- The Gemini API is a web API that allows you to generate text using the Gemini model.
- First, we need to obtain a Gemini API key:
 1. Open Google AI Studio and sign in.
 2. Use **Get API key** → **Create API key**.
 3. Copy the key (a long alphanumeric string).
- Practice good **key hygiene**:
 - **Never** commit API keys in source files, notebooks, or screenshots you share.
 - Prefer a small JSON file that is listed in **.gitignore** (e.g., in your **/workspace/secrets/** folder), or the **GEMINI_API_KEY** environment variable.

- The Gemini API is a web API that allows you to generate text using the Gemini model.
- First, we need to obtain a Gemini API key:
 1. Open Google AI Studio and sign in.
 2. Use **Get API key** → **Create API key**.
 3. Copy the key (a long alphanumeric string).
- Practice good **key hygiene**:
 - **Never** commit API keys in source files, notebooks, or screenshots you share.
 - Prefer a small JSON file that is listed in **.gitignore** (e.g., in your **/workspace/secrets/** folder), or the **GEMINI_API_KEY** environment variable.
 - If a key is ever compromised, **revoke it** and create a new one.

- The Gemini API is a web API that allows you to generate text using the Gemini model.
- First, we need to obtain a Gemini API key:
 1. Open Google AI Studio and sign in.
 2. Use **Get API key** → **Create API key**.
 3. Copy the key (a long alphanumeric string).
- Practice good **key hygiene**:
 - **Never** commit API keys in source files, notebooks, or screenshots you share.
 - Prefer a small JSON file that is listed in **.gitignore** (e.g., in your **/workspace/secrets/** folder), or the **GEMINI_API_KEY** environment variable.
 - If a key is ever compromised, **revoke it** and create a new one.
- There is a Python package called **google-genai** that makes it easy to use the Gemini API.

- The Gemini API is a web API that allows you to generate text using the Gemini model.
- First, we need to obtain a Gemini API key:
 1. Open Google AI Studio and sign in.
 2. Use **Get API key** → **Create API key**.
 3. Copy the key (a long alphanumeric string).
- Practice good **key hygiene**:
 - **Never** commit API keys in source files, notebooks, or screenshots you share.
 - Prefer a small JSON file that is listed in **.gitignore** (e.g., in your **/workspace/secrets/** folder), or the **GEMINI_API_KEY** environment variable.
 - If a key is ever compromised, **revoke it** and create a new one.
- There is a Python package called **google-genai** that makes it easy to use the Gemini API.
- **But we don't do easy here.**

A key loader function

```
import os
import json
from pathlib import Path

def load_api_key(key_file: Path) -> str:
    env_key = os.environ.get("GEMINI_API_KEY", "").strip()
    if env_key:
        return env_key
    if key_file.is_file():
        return json.loads(key_file.read_text())["api_key"]
    else:
        raise FileNotFoundError(f"Key file not found: {key_file}")
    raise ValueError("GEMINI_API_KEY is not set and key file not found.")
```


The REST API is defined at <https://ai.google.dev/gemini-api/docs#rest>.

POST request to Gemini API

The REST API is defined at <https://ai.google.dev/gemini-api/docs#rest>.

Python translation, first set the endpoint:

POST request to Gemini API

The REST API is defined at <https://ai.google.dev/gemini-api/docs#rest>.

Python translation, first set the endpoint:

```
import json
import urllib.request

url = (
    "https://generativelanguage.googleapis.com/v1beta/models/"
    "gemini-2.5-flash:generateContent"
```

Encode the body as JSON and then as bytes:

POST request to Gemini API

Encode the body as JSON and then as bytes:

```
body = {  
    "contents": [  
        {"parts": [{"text": "Explain diffusion models in cognitive science in"}  
    ]  
}  
body_bytes = json.dumps(body).encode("utf-8")
```

Create the request object with the headers:

POST request to Gemini API

Create the request object with the headers:

```
req = urllib.request.Request(  
    url,  
    data=body_bytes,  
    headers={  
        "Content-Type": "application/json",  
        "x-goog-api-key": "YOUR_API_KEY_HERE",  
    },  
    method="POST"  
)
```

Send the request and read the response:

Send the request and read the response:

```
with urllib.request.urlopen(req) as resp:
    response_data = json.loads(resp.read().decode("utf-8"))

text = response_data["candidates"][0]["content"]["parts"][0]["text"]
print(text)
```

Send the request and read the response:

```
with urllib.request.urlopen(req) as resp:
    response_data = json.loads(resp.read().decode("utf-8"))

text = response_data["candidates"][0]["content"]["parts"][0]["text"]
print(text)
```

If `candidates` is empty or missing, the request may have been blocked or malformed.

The `GeminiSimpleAPI` class (`files/gemini_simple_api.py`) implements this pattern.

The `GeminiSimpleAPI` class (`files/gemini_simple_api.py`) implements this pattern.

It wraps the same pieces we just walked through:

The `GeminiSimpleAPI` class (`files/gemini_simple_api.py`) implements this pattern.

It wraps the same pieces we just walked through:

- load an API key from `GEMINI_API_KEY` or a local JSON file

The `GeminiSimpleAPI` class (`files/gemini_simple_api.py`) implements this pattern.

It wraps the same pieces we just walked through:

- load an API key from `GEMINI_API_KEY` or a local JSON file
- build the model endpoint URL

The `GeminiSimpleAPI` class (`files/gemini_simple_api.py`) implements this pattern.

It wraps the same pieces we just walked through:

- load an API key from `GEMINI_API_KEY` or a local JSON file
- build the model endpoint URL
- create JSON request bodies

The `GeminiSimpleAPI` class (`files/gemini_simple_api.py`) implements this pattern.

It wraps the same pieces we just walked through:

- load an API key from `GEMINI_API_KEY` or a local JSON file
- build the model endpoint URL
- create JSON request bodies
- send POST requests with `urllib.request`

The `GeminiSimpleAPI` class (`files/gemini_simple_api.py`) implements this pattern.

It wraps the same pieces we just walked through:

- load an API key from `GEMINI_API_KEY` or a local JSON file
- build the model endpoint URL
- create JSON request bodies
- send POST requests with `urllib.request`
- parse the response text

Using GeminiSimpleAPI

```
from pathlib import Path
from files.gemini_simple_api import GeminiSimpleAPI

client = GeminiSimpleAPI(
    api_key_file=Path("/workspace/secrets/gemini.json"),
    model="gemini-2.5-flash",
    working_dir=Path("/workspace/gemini-output"),
)
```

Using GeminiSimpleAPI

```
from pathlib import Path
from files.gemini_simple_api import GeminiSimpleAPI

client = GeminiSimpleAPI(
    api_key_file=Path("/workspace/secrets/gemini.json"),
    model="gemini-2.5-flash",
    working_dir=Path("/workspace/gemini-output"),
)
```

The class still uses `GEMINI_API_KEY` first, so the `api_key_file` path can be a fallback.

The helper is designed for code-generation examples where we want files back, not just prose.

The helper is designed for code-generation examples where we want files back, not just prose.

```
paths, notes = client.prompt(  
    prompt="Create a small Python file with a function named square.",  
    verbose=True,  
)
```

The helper is designed for code-generation examples where we want files back, not just prose.

```
paths, notes = client.prompt(  
    prompt="Create a small Python file with a function named square.",  
    verbose=True,  
)
```

The response is constrained to JSON with a **files** list, so the client can write the generated files under **working_dir**.

- Limits change over time; check Google's rate-limit documentation every so often.

- Limits change over time; check Google's rate-limit documentation every so often.
- Typical constraints include **requests per minute (RPM)** and **requests per day (RPD)** per model.

- Limits change over time; check Google's rate-limit documentation every so often.
- Typical constraints include **requests per minute (RPM)** and **requests per day (RPD)** per model.
- Exceeding a limit returns HTTP 429.

- Limits change over time; check Google's rate-limit documentation every so often.
- Typical constraints include **requests per minute (RPM)** and **requests per day (RPD)** per model.
- Exceeding a limit returns HTTP 429.
- The per-minute limit often uses a **rolling 60-second window**.

- Limits change over time; check Google's rate-limit documentation every so often.
- Typical constraints include **requests per minute (RPM)** and **requests per day (RPD)** per model.
- Exceeding a limit returns HTTP 429.
- The per-minute limit often uses a **rolling 60-second window**.
- For our examples here, a lighter model with a higher daily quota is usually enough (e.g., **gemini-2.5-flash-lite**).

- Gemini API quickstart: ai.google.dev/gemini-api/docs/quickstart

- Gemini API quickstart: ai.google.dev/gemini-api/docs/quickstart
- Python `urllib.request`: docs.python.org/3/library/urllib.request.html